

Terraform State:

- State is stored by default in a local file named "terraform.tfstate" (json file), but it can also be stored remotely, which works better in a team environment
- There is also a backup of the previous state in the terraform.tfstate.backup
- Terraform uses this local state to create plans and make changes to your infrastructure.
- Prior to any operation, Terraform does a refresh to update the state with the real infrastructure.

Can Terraform state files be stored in a Version Control System (VCS) like Github / GitLab?

What are the potential issues with storing Terraform state files in VCS like Git?

Technically, Terraform state files can be stored in a Version Control System (VCS) like Git, but it is strongly not recommended due to several serious concerns such as.

- **Concurrency Issues:** if multiple users apply changes at the same time it can lead to Inconsistent Infrastructure.
- **Merge conflicts:** State files change frequently, leading to merge conflicts in Git.
- **Security Risks:** State files may contain sensitive data (e.g., access keys, passwords), which could be exposed if the repository is compromised

What are the recommended best practices for managing Terraform state files?

Instead of storing Terraform state files in VCS like GitHub / GitLab, we can use **backend** which will help in better managing state files if we are working in a team of Devops engineers.

What is a Backend in Terraform?

Backend is the mechanism used to **store and manage the Terraform state file** and coordinate Terraform operations.

- Backend keeps **Terraform state file in *sync*** so that multiple users can work parallelly **without any conflicts & without corrupting the state file.**
- Since backend is remote is stored remotely it will ensure all users will always have the latest version of the state
- It avoids multiple commit & push again & again
- Terraform state can be saved remotely using any of below options.
 - s3 bucket
 - azure blob storage
 - consul

Ex5. Write an terraform configuration file to Create a S3 bucket in North Virginia region

```
provider "aws" {  
    region = "us-east-1"  
}  
resource "aws_s3_bucket" "my-first-s3-bucket-name" {  
    bucket = "acecloud-demo-s3bucket-01"  
    #Note: bucket name should be unique across all aws accounts  
}
```

Doc: https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/s3_bucket

Plan apply & observe the resources getting created & check state file getting created locally

Ex6. Write an terraform configuration file to Create an EC2 instance in North Virginia region Use remote backend s3 to store the state file (use s3 bucket created previous exercise)

Write **main.tf** define ec2 instance creation config

```
provider "aws" {  
    region = "us-east-1"  
}  
resource "aws_instance" "ec2-tf-ref-name" {  
    ami = "ami-0c7af5fe939f2677f"  
    instance_type = "t2.micro"  
    tags = {  
        Name = "rahul-instance"  
    }  
}
```

Also create **backend.tf** file to store state file remotely in s3

```
terraform {  
    backend "s3" {  
        bucket = "acecloud-demo-s3bucket-01" #S3 bucket name  
        key    = "rahul-state-file/terraform.tfstate" #path inside your s3 bucket  
        region = "us-east-1"  
    }  
}
```

Doc: <https://developer.hashicorp.com/terraform/language/backend/s3>

Run terraform init & observe the backend is getting picked as s3

Run plan & apply & observe state file not getting created locally

go to bucket & check state file got created remotely as backend.

Now multiple users can work parallelly & easily without corrupting state file & infrastructure.

Other advantages of using backend

- Automatic state locking to prevent concurrent modifications.
- we can encrypt all data inside the state file.
- we can control on who can access to the state file.

What is state file locking?

State file locking ensures that only one process or person can modify the state at a time, preventing conflicting changes to infrastructure.

How to lock state file in s3 backend?

dynamodb_table is used to manage state locks to prevent concurrent changes.

We can add this in backend.tf file like below

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state"
    key         = "path/to/statefile.tfstate"
    region      = "us-west-2"
    dynamodb_table = "terraform-lock-table" # DynamoDB table for state locking
    encrypt     = true
  }
}
```